

```
>> /etc/apt/sources.list.d/varnish-cache.list
apt-get update
apt-get install varnish
```

With this, Varnish is already running on your server and has started to cache. The Varnish configuration file is generally located at `/etc/varnish/default.vcl`.

Varnish has several built-in sub-routines, which are called the several stages of the caching fetch process. We can also define custom sub-routines, which can be called within these built-in sub-routines. The following are the built-in sub-routines for Varnish.

- `vcl_init` – Called every time before the configuration file is loaded
- `vcl_fini` – Called every time after the configuration file is discarded
- `vcl_recv` – Called when a new HTTP request is received by Varnish
- `vcl_pipe` – Called when the request is sent to the backend
- `vcl_pass` – Called before delivery, when the request is coming from the backend fetch
- `vcl_hit` – Called when the request is found in the Varnish cache
- `vcl_miss` – Called when the request is not found in the cache before forwarding to the backend
- `vcl_hash` – Called after the hash is created for the received request
- `vcl_purge` – Called when the cache is purged for the request
- `vcl_deliver` – Called when the output is delivered by Varnish
- `vcl_backend_fetch` – Called before fetching a request from the backend
- `vcl_backend_response` – Called after the response from the backend is received by Varnish
- `vcl_backend_error` – Called when fetching from backend fails after `max_retries` attempts

These sub-routines can be used in the VCL

configuration file to perform the desired actions at various stages. This gives us high flexibility for customisation in Varnish. We can also check the syntactical correctness of the configuration file using the following command:

```
varnishd -C -f /etc/varnish/default.vcl
```

Varnish gives a detailed description of any error in the syntax, similar to what is available with NGINX and Apache servers.

Performance and benchmarking

To see the actual difference in performance, we have used the Apache Benchmark tool, which is available with the `apache2-utils` package. For our tests, we have hosted a

```
shin@shinbhat:~$ ab -n 100 -c 10 http://mywebsite.com/
This is ApacheBench, Version 2.3 <http://www.apache.org>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking mywebsite.com (be patient).....done


Server Software:      Varnish
Server Hostname:      mywebsite.com
Server Port:          80

Document Path:        /
Document Length:       137762 bytes

Concurrency Level:     10
Time taken for tests:   0.344 seconds
Complete requests:     100
Failed requests:        0
Total transferred:      13870400 bytes
HTML transferred:       13776200 bytes
Requests per second:    294.00 [#/sec] (mean)
Time per request:       34.388 [ms] (mean)
Time per request:       34.383 [ms] (mean, across all concurrent requests)
Transfer rate:          3438.23 [Kbytes/sec] received

Connection Times (ms)
  min  mean[+sd] median max
Connect:  0  1  0.2  1  3
Processing:  4  33  68.9  8  344
Waiting:  0  25  38.3  2  240
Total:  4  38  69.8  11  344

Percentage of the requests served within a certain time (ms)
 50%  11
 60%  13
 70%  14
 80%  15
 90%  20
 95%  28
 98%  282
 99%  282
 99.5% 284
 100% 304 (longest request)
shin@shinbhat:~$
```

Figure 1: Benchmarking with Varnish

```
shin@shinbhat:~$ ab -n 100 -c 10 http://mywebsite.com/
This is ApacheBench, Version 2.3 <http://www.apache.org>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking mywebsite.com (be patient).....done


Server Software:      nginx
Server Hostname:      mywebsite.com
Server Port:          80

Document Path:        /
Document Length:       137762 bytes

Concurrency Level:     10
Time taken for tests:   0.344 seconds
Complete requests:     100
Failed requests:        0
Total transferred:      13870400 bytes
HTML transferred:       13776200 bytes
Requests per second:    294.00 [#/sec] (mean)
Time per request:       34.388 [ms] (mean)
Time per request:       34.383 [ms] (mean, across all concurrent requests)
Transfer rate:          3438.23 [Kbytes/sec] received

Connection Times (ms)
  min  mean[+sd] median max
Connect:  0  1  0.2  1  3
Processing:  4  33  68.9  8  344
Waiting:  0  25  38.3  2  240
Total:  4  38  69.8  11  344

Percentage of the requests served within a certain time (ms)
 50%  11
 60%  13
 70%  14
 80%  15
 90%  20
 95%  28
 98%  282
 99%  282
 99.5% 284
 100% 304 (longest request)
shin@shinbhat:~$
```

Figure 2: Benchmarking with NGINX (without Varnish)

fully loaded WordPress site on a `t2.micro` instance of EC2 in AWS. Let's call it `mywebsite.com` in our local host file to avoid DNS resolution delays in our tests. This server runs Varnish on Port 80 and the NGINX server on Port 8080.

The syntax to run the test is `ab -n <num_requests> -c <concurrency> <addr>[:<port>] <path>`

Figures 1 and 2 show the statistics on running 100 requests with 10 concurrent threads.

Considering the benchmarking result, we get the mean time spent per request with Varnish as 3.438 ms, and without Varnish as 57.999 ms. It is evident that Varnish is a winner and a must-have tool for your Web servers, allowing you to boost the performance up to 1000x, depending on your configurations and architecture. 

By: Krishna Modi

The author has a B. Tech degree in computer engineering from NMIMS University, Mumbai and an M. Tech in cloud computing from VIT University, Chennai. He has rich and varied experience at various reputed IT organisations in India. He can be reached at krish512@hotmail.com.